

# Test Task: Order Processing Service

**Stack:** Python 3.12+, FastAPI, SQLAlchemy 2.x async, PostgreSQL, Kafka, Alembic, pytest, Docker Compose.

## Requirements

Implement products and orders.

**API:** POST /products. GET /products. POST /orders. GET /orders/{id}. GET /orders.

```
{ "customer_id": "uuid", "items": [{"product_id": "uuid", "quantity": 2}] }
```

- Validate that products exist and enough stock is available.
- Decrease stock safely under concurrent requests.
- Calculate order total using Decimal.
- Store product price in order\_items as a snapshot.
- Support Idempotency-Key so duplicate requests create only one order.
- Create an order.created Kafka event.

## Kafka

Use the **Transactional Outbox Pattern** so the order and outgoing event are stored in the same PostgreSQL transaction.

- Create an outbox worker that publishes to orders.events.
- Create a consumer for order.created that writes a notification record.
- Treat Kafka delivery as **at-least-once**. The consumer must be idempotent, so the same event\_id cannot create more than one notification.

## Important Cases

- Concurrent orders for the last available product.
- Duplicate Idempotency-Key.
- Duplicate Kafka events.
- Database rollback.
- Producer/consumer retries.

## Tests

At minimum cover: successful order creation, insufficient stock, nonexistent product, correct total calculation, concurrent stock update, duplicate idempotency key, duplicate Kafka event, and outbox event creation together with the order.

## Delivery

The project should start with `docker compose up`.

Include a short README describing the project structure, how to run it, how concurrency is handled, and how Kafka delivery/idempotency are handled.

**Bonus:** FOR UPDATE SKIP LOCKED for multiple outbox workers, retry strategy, Kafka DLQ.